```
$myArr = ["Bob", "Russell"];
array_unshift($myArr, "Alice");
print_r($myArr);
```

```
Output:
Array
(
    [0] => Alice
    [1] => Bob
    [2] => Russell
)
```

As promised, Alice has been added to `$myArr` as the first element. Notice that `$myArr` has also been reindexed, and the original elements now have new keys. `array_unshift()` has a counterpart – `array_shift()` – which is used for removing the first element, and we'll look at that shortly.

Inserting into Arbitrary Positions

Appending and prepending new elements is straightforward, as we have seen. What if we need to insert new elements in between existing ones? Fortunately, PHP's array toolbox has just the function we need: `array_splice()`. This function has a range of uses (the official PHP documentation states that it is used to "remove a portion of the array and replace it with something else"), but one of the parameters tells the function how many elements need to be removed, and that value can be zero.

This makes `array_splice()` useful for inserting new elements in arbitrary positions. The function signature is `array_splice(array &$original, int $offset, int $length, mixed $replacement)`. `$offset` is the starting position of the original array that we wish to change, `$length` means the number of elements to remove, `$replacement` is what, if anything, we'd like to insert (this can be a scalar or composite type):

```
$myArr = ["Rita", "Bob"];
array_splice($myArr, 1, 0, "Sue");
print_r($myArr);
```